Glacier Quest

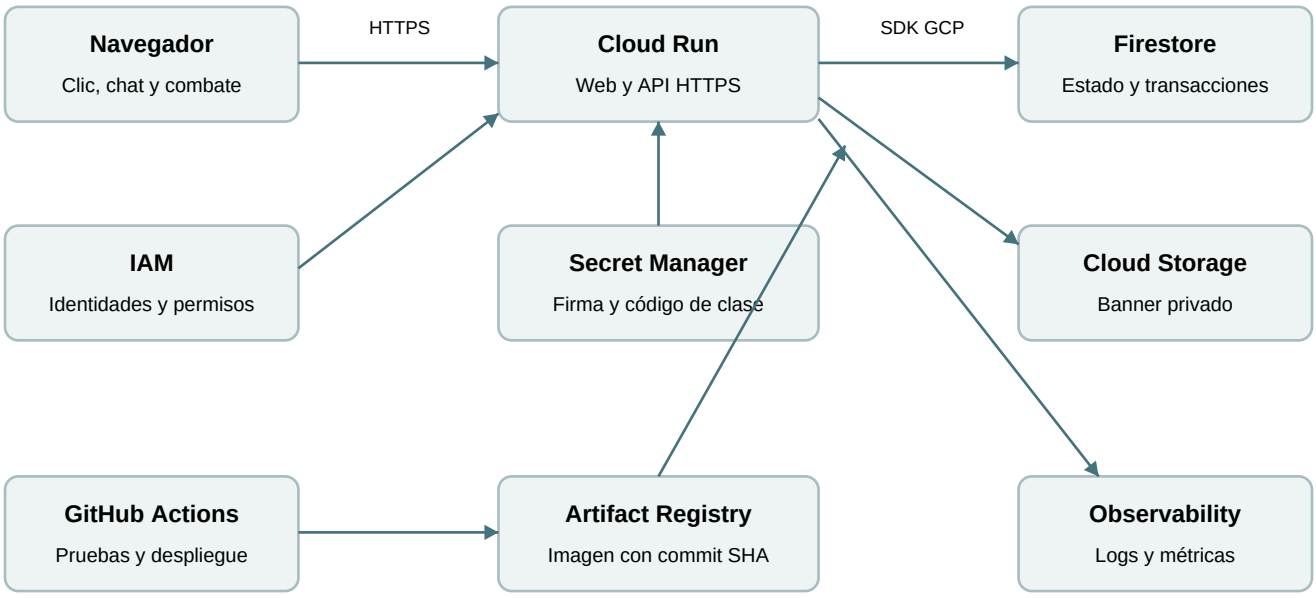
Avance de arquitectura y servicios

Cloud Computing 2026 · Universidad Galileo · Iteración 0.2 · 7 de octubre de 2026

El avance ofrece una plaza multijugador sencilla con movimiento por clic, chat, objetos y combate por turnos. La prioridad es que cada servicio tenga una función visible y una evidencia verificable. Se propone una arquitectura de ocho servicios, sin instancia SQL ni balanceador con costo fijo.

El juego funciona localmente y Terraform valida. El despliegue real sigue pendiente porque todavía no se dispone del proyecto GCP, facturación ni repositorio del equipo. M2 exige al menos dos servicios desplegados para el 9 de octubre; la demo local por sí sola no satisface ese requisito.

Arquitectura propuesta



Logging y Monitoring se contabilizan juntos. El diagrama representa la arquitectura preparada; no certifica que los recursos estén activos.

Decisiones que reducen costo

Firestore Native vía GCP reemplaza PostgreSQL en Cloud SQL. La regla general del curso sí lo permite; la prohibición indicada en el playbook era incorrecta. Cloud Run sirve web y API con HTTPS. Storage conserva un recurso visual privado. Se retiran Cloud CDN, su balanceador y la IA opcional para concentrarse en integración, evidencia y presupuesto.

Servicios y pruebas de integración

Cada servicio debe demostrarse mediante una acción del juego o una evidencia de operación. Un recurso vacío o una plantilla no equivalen a una integración.

Servicio	Responsabilidad	Evidencia
Cloud Run	Hospeda web y API Express. Escala de cero a dos instancias.	URL, revisión y /health.
Firestore Native	Inventario, salud, monedas, duelo, chat y presencia. Transacciones evitan duplicar turnos.	Documento cambia tras una acción y persiste al recargar.
Cloud Storage	Banner privado descargado por la API con SDK GCP. Caché de cinco minutos.	Objeto en bucket e imagen en juego. Sin fallback silencioso en nube.
IAM	Runtime y deploy separados. Acceso por recurso; federación limitada a GitHub main.	Bindings y cuentas de servicio configuradas.
Secret Manager	Clave HMAC de sesión y código de clase; versiones explícitas.	Referencias de secretos en revisión, sin exponer valores.
Observability	Logs JSON con requestId y panel de métricas nativas de peticiones y latencia.	Solicitud localizada en Logging y panel con tráfico.
GitHub Actions	Pruebas y despliegue manual de imagen mediante credenciales temporales.	Ejecución exitosa y revisión actualizada.
Artifact Registry	Imágenes privadas versionadas por SHA y limpieza de versiones antiguas.	Digest de imagen utilizado por Cloud Run.

Son ocho servicios planificados, con Logging y Monitoring como uno. El curso menciona categorías abiertas de seguridad y entrega; confirmar con el docente si pide una clasificación adicional para Secret Manager o Artifact Registry.

Juego y alcance de la iteración

Movimiento y sala

El jugador elige destino con clic o toque. El movimiento local es continuo; los demás jugadores se actualizan cada cinco segundos y se interpolan. Las pestañas ocultas dejan de consultar. Las flechas funcionan como alternativa cuando el mapa está enfocado, sin interferir con el chat.

Combate con decisiones

El guardián repite golpe de hielo, carga y ventisca. Atacar causa 8, o 12 durante su carga. Defender reduce el daño un 75 %, con redondeo hacia arriba; la ventisca de 14 baja a 4. Hay dos pociones de 12 PV; curar consume turno. Huir no entrega premio. Ganar otorga diez monedas una sola vez. Un nuevo duelo restaura salud tras un descanso de diez segundos.

Persistencia y sesiones

Cada jugador recibe UUID y sesión firmada. Un nombre repetido crea otro jugador; no permite tomar el inventario de otra persona. La misma pestaña conserva la sesión al recargar. Firestore guarda inventario y duelo mediante transacciones. Los objetos tienen IDs únicos por jugador; repetir una recolección no entrega más monedas.

Pruebas verificadas

Siete pruebas de lógica y API aprobadas. La carga local de treinta jugadores completó 360 sincronizaciones sin errores, con p95 de 117 ms. Se comprobó en navegador el ingreso, movimiento por clic, ataque, contraataque y defensa. Terraform 1.12.2 con proveedor Google 7.46.1 aprobó formato y validate. La auditoría npm no reportó vulnerabilidades tras actualizar dependencias indirectas.

Verificaciones pendientes

No se ejecutó un despliegue de nube ni un build Docker en esta máquina. El emulador oficial de Firestore no arrancó por una falla de Java con conexiones internas en Windows; la prueba de persistencia concurrente queda incluida para ejecutarla en un entorno compatible. Ninguna prueba local sustituye la comprobación de IAM y persistencia en GCP.

Límites aceptados

La sala muestra hasta cuarenta presencias y está pensada para treinta personas. El chat conserva treinta mensajes en un documento. No hay recuperación de cuentas, limpieza automática ni antitrampas de movimiento. Se usa código de clase para reducir acceso casual. La posición visible es transitoria; se guardan posiciones en eventos de inventario o combate.

Estimación de costos

Fecha de consulta: 6 de octubre de 2026, USD, región us-central1. Estimación de planificación, no cotización ni garantía de factura cero. No depende de créditos de bienvenida. Las cuotas pueden compartirse con otros proyectos o recursos de la cuenta.

Escenario académico

Diez sesiones al mes de una hora, seis jugadores simultáneos y una demostración adicional de una hora con treinta jugadores. Polling cada cinco segundos, pestañas cerradas fuera de sesiones, 100 combates/mes y chat moderado. Por sesión pequeña: 6 × 720 = 4.320 solicitudes; cada sync lee 1 jugador + hasta 6 presencias + 1 documento de chat y escribe 1 presencia. Resultado: unas 34.560 lecturas y 4.320 escrituras por día de ensayo, más acciones y reintentos. Diez ensayos deben repartirse en días distintos para aplicar la cuota diaria como se estima.

Demo de 30 jugadores: 21.600 sync/h; hasta 32 lecturas/sync = 691.200 lecturas/h y 21.600 escrituras/h. Por tanto, el polling NO es gratis ilimitadamente. Con 50.000 lecturas y 20.000 escrituras gratuitas diarias disponibles, exceso aproximado de 641.200 lecturas y 1.600 escrituras. A USD 0,03/100.000 lecturas y USD 0,09/100.000 escrituras en Iowa, unos USD 0,194 para esa hora, antes de acciones, reintentos, almacenamiento y red. Redondear a USD 0,25 para operaciones de base de datos de esa demo. El conteo no incluye lecturas de consola ni reintentos de transacciones.

Servicio	Supuesto y cuota relevante	Estimación mensual
Cloud Run	~65.000 sync más acciones/estáticos; 11 horas × 2 instancias × 1 vCPU = 79.200 vCPU-s; 39.600 GiB-s a 512 MiB; debajo de cuotas si están disponibles	USD 0 de cómputo bajo supuestos; reservar USD 1 para salida y variación
Firestore	Ensayos bajo cuota diaria; demo excede lecturas; menos de 1 GiB total	USD 0,25 aproximados de operaciones; reservar hasta USD 1
Cloud Storage	Un SVG pequeño, lecturas cacheadas, región elegible	Cercano a USD 0 con cuota; reservar USD 0,10
IAM	Roles y federación	Sin cargo directo; productos accedidos se facturan
Secret Manager	2 versiones y menos de 10.000 accesos/mes	USD 0 con cuota disponible
Logging y Monitoring	Menos de 1 GiB logs, métricas nativas, sin alertas métricas ni uptime checks provisionados	USD 0 bajo esas condiciones
GitHub Actions	~20 builds × 5 min = 100 min Linux; depende del plan y consumo restante	USD 0 dentro del cupo; validar en cuenta
Artifact Registry	Ejemplo: 1 GiB-mes total de imágenes, 0,5 GiB gratis, USD 0,10/GiB-mes adicional	USD 0,05; medir tamaño real

Estimación central con cuotas disponibles: ~USD 0,30 más salida de red y variaciones. Presupuesto de trabajo recomendado: USD 1-3 al mes; alertas a USD 5 y USD 10. El presupuesto es una reserva razonable para este uso, no un límite automático ni una promesa. Guatemala puede tener tarifa de salida distinta a la cuota de Cloud Run dentro de Norteamérica: revisar el destino en la calculadora y la factura.

Si dejan treinta jugadores conectados

Una hora diaria durante treinta días implica aproximadamente 20,7 millones de lecturas. El exceso, descontando 50.000 diarias, cuesta alrededor de USD 5,77 solo en lecturas. Mantener dos instancias ocupadas durante 30 horas puede superar la cuota de CPU de Cloud Run. Dejar la app abierta durante días cambia radicalmente el escenario; el máximo de instancias no impide sumar horas o consultas.

Ahorro frente al playbook

Firestore elimina la instancia SQL facturada por tiempo. Retirar Cloud CDN evita el balanceador externo: una regla de reenvío a USD 0,025/h suma USD 18,25 por 730 h, antes de procesamiento, transferencia y CDN. El supuesto de que todo salvo SQL era gratis no era correcto. El límite gratuito de Logging consultado es 50 GiB/proyecto/mes, no los 150 GB indicados en el playbook.

Fuentes oficiales

- Firestore y precio regional: https://cloud.google.com/firestore/pricing
- Cloud Run y transferencia: https://cloud.google.com/run/pricing
- Storage: https://cloud.google.com/storage/pricing
- Secret Manager: https://cloud.google.com/secret-manager/pricing
- Observability: https://cloud.google.com/products/observability/pricing
- Artifact Registry: https://cloud.google.com/artifact-registry/pricing
- Balanceador: https://cloud.google.com/load-balancing/pricing
- GitHub Actions: https://docs.github.com/en/billing/concepts/product-billing/github-actions
- Calculadora para adjuntar estimación del equipo: https://cloud.google.com/products/calculator

Despliegue paso a paso

Estado: código preparado; hace falta el proyecto GCP con facturación y el repositorio de GitHub. No se creó infraestructura ni se gastó crédito en esta iteración. Comandos de nube escritos para Bash en Google Cloud Shell. Para el juego local, ver README.

1 Preparar la cuenta

Crear o seleccionar un proyecto dedicado, habilitar facturación y crear alertas de presupuesto de USD 5 y USD 10. Un presupuesto avisa; no corta el gasto. Elegir `us-central1` para mantener los recursos juntos y usar cuotas gratuitas regionales cuando apliquen. No crear Firebase.

La persona que prepara infraestructura necesita permisos para habilitar APIs, crear Firestore, buckets, secretos, repositorios, Cloud Run, paneles e identidades y configurar sus políticas IAM. Usar una cuenta administradora temporal del proyecto de clase para bootstrap; no asignar Owner o Editor a las cuentas de la aplicación o CI. La organización puede impedir `allUsers`: en ese caso resolver la política con su administrador antes de publicar.

En Cloud Shell, clonar el repositorio y entrar a `glacierquest`. Terraform y Docker deben estar disponibles. Los workflows deben estar en `.github/workflows` de la RAÍZ del repositorio, al mismo nivel que `glacierquest`, no dentro de esa carpeta. Si cambian la estructura, ajustar sus rutas.

```
gcloud config set project YOUR_PROJECT_ID
gcloud auth application-default login
cp infra/terraform/terraform.tfvars.example infra/terraform/terraform.tfvars
```

Editar `terraform.tfvars`: project_id, OWNER/REPOSITORY, ID numérico del repositorio y del propietario. Se obtienen desde `https://api.github.com/repos/OWNER/REPOSITORY` (campos id y owner.id) con acceso autorizado. No son secretos. Mantener `deploy_app=false`. Si ya existe Firestore `(default)`, importarlo antes de aplicar y ajustar región al recurso existente; su ubicación no se cambia en sitio.

```
terraform -chdir=infra/terraform init
terraform -chdir=infra/terraform fmt -check
terraform -chdir=infra/terraform validate
# SOLO si la base ya existe, sustituir el ID:
# terraform -chdir=infra/terraform import google_firestore_database.game
'projects/YOUR_PROJECT_ID/databases/(default)'
terraform -chdir=infra/terraform plan -out=bootstrap.plan
terraform -chdir=infra/terraform apply bootstrap.plan
```

Revisar el plan antes de aplicarlo. Guardar el estado de Terraform fuera de Git y compartirlo con control de acceso. Esta versión usa estado local; para colaborar, migrarlo después a un backend GCS protegido y con versionado. No ejecutar dos apply simultáneamente.

2 Añadir secretos sin guardarlos en Git ni Terraform

Ejecutar una vez después del bootstrap. La clave se genera sin imprimirla. Elegir un código privado para la clase; no reutilizar contraseñas personales. Las versiones nuevas comienzan normalmente en 1; verificar los números devueltos y ajustar `session_secret_version` y `demo_secret_version` en tfvars. No usar `latest` para una rotación silenciosa.

```
openssl rand -hex 32 | tr -d '\n' | gcloud secrets versions add glacierquest-session --data-file=-
read -rs -p 'Código para la clase: ' CLASS_CODE
printf '%s' "$CLASS_CODE" | gcloud secrets versions add glacierquest-demo-code --data-file=-
unset CLASS_CODE
```

3 Construir la imagen y desplegar

Desde `glacierquest`, en Cloud Shell:

```
PROJECT_ID=$(gcloud config get-value project)
REGION=us-central1
IMAGE="$REGION-docker.pkg.dev/$PROJECT_ID/glacierquest/game:initial"
gcloud auth configure-docker "$REGION-docker.pkg.dev" --quiet
npm ci --ignore-scripts --prefix api
npm test --prefix api
docker build -t "$IMAGE" .
docker push "$IMAGE"
```

Editar `terraform.tfvars`: `deploy_app=true`, `image` con la ruta subida y las versiones de secretos correctas. IAM puede tardar unos minutos en propagarse; si hay un 403 al crear la revisión, revisar el binding y reintentar después de su propagación.

```
terraform -chdir=infra/terraform plan -out=app.plan
terraform -chdir=infra/terraform apply app.plan
URL=$(terraform -chdir=infra/terraform output -raw url)
curl --fail "$URL/health"
```

Esperado: status ok, database firestore, cloud true. Abrir URL, introducir código de clase y recoger una moneda. Recargar la pestaña y comprobar el inventario. Revisar Firestore en GCP, el banner en Storage y una petición en Logging. Si una dependencia falla, reparar la conexión; no cambiar a memoria para aparentar un despliegue funcional.

4 Activar CI y CD

Crear el entorno GitHub `class-demo` y estas variables de repositorio o entorno, tomadas de las salidas de Terraform: GCP_PROJECT_ID, WIF_PROVIDER, DEPLOY_SERVICE_ACCOUNT y ASSET_BUCKET. No subir un archivo JSON de clave de cuenta de servicio.

El workflow `Validate Glacier Quest` corre con push/PR y ejecuta pruebas, build Docker y validación Terraform. `Deploy Glacier Quest` se lanza manualmente desde Actions en main; vuelve a probar, obtiene credenciales temporales por OIDC, sube imagen con SHA, actualiza el banner y la revisión existente, y verifica `/health`. El despliegue manual evita publicar cada experimento o gastar minutos de CI innecesarios. Habilitar aprobación del entorno si el equipo la necesita.

Terraform ignora cambios de la imagen después de crear Cloud Run para que un apply de infraestructura no revierta el despliegue de Actions. Para rollback, elegir la revisión anterior en Cloud Run o reasignar tráfico explícitamente. Conservar una imagen conocida y verificar salud después de revertir. Cambiar la clave de firma invalida sesiones existentes; planificar esa rotación fuera de la presentación.

5 Evidencias para M2

Capturar URL, proyecto seleccionado, revisión Cloud Run y `/health`. Capturar el documento Firestore antes y después de recolectar o ganar. Adjuntar diagrama y lista de servicios con estado real. Esto prueba al menos cómputo y base de datos. No mostrar valores de secretos ni tokens del navegador. Guardar fecha, commit y responsable de cada prueba en `docs/evidence.md`.

6 Cierre y costos

Cerrar las pestañas al terminar: una pestaña visible sigue sincronizando aunque el jugador esté quieto. Ocultarla pausa el polling. Cloud Run puede bajar a cero, pero esto no es un tope monetario. No configurar uptime checks que despierten la app cada minuto sin necesidad. La política del registro retiene tres versiones recientes y elimina las viejas de más de siete días; vigilar su almacenamiento.

Antes de retirar el proyecto, exportar lo que deban conservar. `terraform destroy` no es un botón de apagado: Firestore tiene protección y política ABANDON; un bucket no vacío tampoco se elimina

automáticamente. Revisar y borrar esos recursos de forma deliberada tras la calificación. No borrar el proyecto antes de entregar el video y la URL.

Seguridad y límites conocidos

La API toma el jugador de una sesión HMAC, nunca de un playerId enviado por el cliente. Nombres iguales no dan acceso al mismo inventario. Las sesiones caducan en siete días; no hay recuperación de cuentas. En local la clave cambia al reiniciar. En nube la clave y el código de clase vienen de Secret Manager, con versiones explícitas. Código compartido es una barrera para la demo, no autenticación institucional.

Inventario y turnos viven en transacciones Firestore por jugador. Cada turno exige ID de batalla y número actual: una repetición o clic concurrente no duplica recompensa. Cada objeto tiene ID fijo y se recoge una vez por jugador. Los mensajes se insertan como texto en el DOM. La API limita tamaños, valida tipos y coordenadas y no devuelve trazas internas.

El bucket es privado, con prevención de acceso público. Cloud Run lee el objeto usando roles/storage.objectViewer. La cuenta runtime tiene roles/datastore.user y acceso solo a los dos secretos; no administra infraestructura. El alcance de datastore.user es el proyecto: por eso se recomienda proyecto dedicado. Las reglas Firebase no protegen el SDK servidor; la protección relevante aquí es IAM y la validación de API.

La cuenta de deploy puede escribir en ese registro y bucket, actuar como runtime y actualizar ese servicio. La federación solo admite el ID numérico del repositorio, propietario y refs/heads/main. Proteger main y las modificaciones de workflows; quien despliega código puede hacer que runtime use sus permisos. No asignar claves JSON persistentes ni exponer secretos en variables de GitHub.

Limitaciones: coordenadas controladas por cliente, límite de solicitudes por instancia, máximo de cuarenta presencias, chat concentrado en un documento y registros antiguos sin limpieza automática. No anunciar resistencia a abuso público ni a tráfico masivo. El código de clase y escala máxima ayudan a acotar uso académico; un presupuesto solo notifica. Antes de una publicación abierta harían falta autenticación recuperable, protección distribuida, cuotas por usuario y reglas de movimiento más estrictas.

Los logs incluyen ruta sin query, estado, duración y requestId; no incluyen tokens, nombres ni mensajes. La UI comprueba base y objeto, pero indica revisión manual para IAM, Secret Manager, CI y Observability. Mostrar un estado de consola verificado es responsabilidad de la demostración.

Registro de evidencias

Verificado localmente en esta iteración

• Siete pruebas de lógica/API aprobadas: contraataque, vulnerabilidad, defensa, curación, derrota, repetición de turno, recolección, sesiones, chat y errores.

• Simulación con 30 jugadores, 360 sync durante un minuto: sin errores; p95 de 117 ms. Backend en memoria. No extrapolar a Firestore ni a red de nube.

• Navegador: ingreso, movimiento por clic, acercamiento al guardián, ataque y contraataque visibles.

• Dependencias: auditoría npm sin vulnerabilidades reportadas después de actualizar proxy-addr y uuid mediante overrides. Es una revisión de dependencias, no una auditoría de seguridad completa.

• Terraform: formato y validate aprobados con Terraform 1.12.2 y proveedor Google 7.46.1. No se ejecutó plan contra una cuenta ni apply.

Evidencia de nube pendiente

Verificación	Estado	Evidencia que debe adjuntar el equipo
Proyecto con facturación	Pendiente	ID, región, captura sin datos sensibles
Cloud Run y Firestore conectados	Pendiente	URL y respuesta /health; documento persistido
Storage privado	Pendiente	Objeto, imagen en juego y prueba de lectura vía API
IAM y Secret Manager	Pendiente	Roles y referencias de versiones, nunca valores
CI/CD	Pendiente	URL de ejecución exitosa, commit y revisión
Observability	Pendiente	RequestId en logs y panel con datos
Carga real con 30 jugadores	Pendiente	Fecha, versión, región, p95 y fallos
Docker en entorno de despliegue	Pendiente	Build y arranque con variables de producción

Completar cada fila solo después de ejecutarla. M2 requiere dos servicios desplegados, no dos pruebas locales.

Guion de video de seis minutos

0:00-0:40: objetivo y diagrama, prioridad de integración de servicios. 0:40-1:30: dos navegadores, movimiento por clic y chat. 1:30-2:20: recoger objeto, recargar y mostrar el documento de Firestore. 2:20-3:10: duelo, anuncio de ventisca, defensa y recompensa persistente. 3:10-4:10: bucket privado, referencia de secreto y cuenta runtime. 4:10-5:10: ejecución de Actions, imagen y revisión desplegada;

requestId en Logging y métricas. 5:10-6:00: costo mensual, limitaciones y siguiente trabajo. No declarar completada ninguna evidencia pendiente.

Referencias del proyecto

Reglas: _2026__Course_Project.docx. Plan inicial: Glacier_Quest_Project_Playbook.docx. El informe conserva el objetivo del plan, pero corrige la restricción de Firestore y la estimación de costos fijos. La referencia Roastfolio proporcionada se conserva como inspiración; no se utilizó como evidencia técnica porque no fue posible consultar su contenido.

El código y las guías Markdown acompañan el PDF. Los comandos largos pueden aparecer partidos visualmente en el informe; para copiarlos y ejecutarlos, utilizar docs/deployment-steps.md. No se modificaron los documentos originales.